

Star Yarbrough
ITAI 2373 – Natural Language Processing
Date 08/29/26

Lab 02 Reflection Journal

One of the biggest things I learned from this lab is that text preprocessing is not as simple as just “cleaning up” text. Before working through these examples, I assumed that the cleaner and more standardized the text was, the easier it would be for an NLP system to understand. What surprised me was learning that too much preprocessing can actually remove information that the system needs. An exclamation point, emoji, hashtag, or even a common word may seem unnecessary, but depending on the purpose of the system, removing it could change the meaning of the text.

The social media examples made this especially clear to me. Something like “SO GOOD!!!” expresses more emotion than simply saying “good.” Emojis such as 🥰 or 👍 also communicate feelings without using words at all. If I were analyzing customer reviews or social media posts, removing those features could make it harder to accurately determine how someone feels. This made me realize that preprocessing has to be designed around the goal of the NLP system rather than using the same cleaning process for everything.

I also found the comparison between NLTK and spaCy interesting. At first, I expected one library to simply be better than the other, but that was not really what I observed. They sometimes handled the same text similarly, but spaCy provided additional information about the words, such as their parts of speech and lemmas. At the same time, there were small situations where NLTK handled something differently, such as keeping the coffee emoji together while spaCy split it. Their stop word lists were also surprisingly different. This showed me that even commonly used NLP tools can make different decisions about the exact same text.

The difference between stemming and lemmatization was probably one of the most memorable parts for me. Some of the stemming results looked strange, such as “flying” becoming “fli,” “was” becoming “wa,” and “battery” becoming “batteri.” Lemmatization was able to recognize more meaningful relationships, such as “was” becoming “be,” “children” becoming “child,” and “feet” becoming “foot.” Seeing the actual results helped me understand why lemmatization can be more useful when preserving meaning is important, even though stemming may be faster.

This lab also made me think differently about problems I have experienced with real language-processing systems. For example, Siri sometimes listens to my command in the car but does not actually send the text or make the call I requested. My Google Home has also started listening after hearing me say “okay” in a normal conversation, even though I never said “Okay Google.” Even conversational AI can sometimes focus on one part of what someone says while missing

the overall point. These experiences make more sense to me now because processing human language involves much more than simply recognizing individual words.

One question I still have is how developers determine exactly how much preprocessing is enough. There seems to be a point where removing noise improves the data, but removing too much starts destroying useful information. I am also curious about how these traditional preprocessing techniques are used with newer AI language models and whether modern models need the same amount of preprocessing.

In future projects, I think the most important lesson I will carry with me is to choose preprocessing techniques based on the specific problem instead of automatically applying every available technique. If I were working with customer reviews, chatbots, or social media sentiment, I would want to preserve more of the language that communicates emotion and context. For search or keyword matching, I might prioritize speed and reducing word variations instead. This lab helped me understand that good preprocessing is not about creating the cleanest text possible; it is about creating the most useful text for the problem being solved.